

Т3: Python-утилита мониторинга учебной деятельности

1. Назначение

Утилита должна помогать отслеживать, сколько времени пользователь проводит в профильных приложениях во время обучения.

Примеры приложений:

```
VSCode
PyCharm
Windows Terminal
PowerShell
Browser
VMware / VirtualBox
GNS3
Obsidian
DBeaiver
Postman
```

Главная идея:

Приложение активно → время считается → после завершения/смены активности пользователь выбирает навык → данные сохраняются

Пример:

```
VSCode был активен 47 минут
Утилита спрашивает:
"Какой навык прокачивал?"

Пользователь выбирает:
Python / FastAPI / Backend

В базу сохраняется учебная сессия.
```

2. Важное ограничение

Утилита должна быть предназначена только для личного использования на собственном ПК.

Не нужно делать:

скрытый запуск
слежку за другими пользователями
отправку данных на сторонние серверы
перехват клавиатуры
перехват паролей
скриншоты экрана
запись содержимого окон

Разрешено:

получать имя активного процесса
получать заголовок активного окна
считать длительность активности
сохранять локальную историю учебных сессий
отправлять данные в свой backend

3. Цель MVP

Сделать простую Python-программу, которая:

1. Отслеживает активное окно Windows.
2. Определяет имя процесса.
3. Считает, сколько времени окно было активно.
4. Если активность длилась больше заданного порога, предлагает выбрать навык.
5. Сохраняет результат в локальный JSON или SQLite.

4. Платформа MVP

Первая версия:

Windows 11
Python 3.11+
CLI-интерфейс
Локальное хранение данных

Позже:

GUI
FastAPI integration
SQLite/PostgreSQL
Экспорт в dashboard

5. Рекомендуемый стек

Базовая версия

```
Python  
pywin32  
psutil  
sqlite3 или json
```

Для CLI

```
rich  
typer
```

Для GUI позже

```
PySide6  
или  
customtkinter
```

Для отправки в backend позже

```
requests  
или  
httpx
```

6. Что должна фиксировать утилита

ActivityEvent

```
{  
  "id": 1,  
  "started_at": "2026-05-21T19:30:00",  
  "ended_at": "2026-05-21T20:10:00",  
  "duration_seconds": 2400,  
  "process_name": "Code.exe",  
  "window_title": "main.py - Resume Progress Platform - Visual Studio Code",  
  "skill": "Python",  
  "activity_type": "coding",  
  "note": "Писал FastAPI endpoint для skills"  
}
```

7. Основные сущности

7.1 AppActivity

Сырые данные об активности приложения.

```
id
process_name
window_title
started_at
ended_at
duration_seconds
```

7.2 LearningSession

Осмысленная учебная сессия, привязанная к навыку.

```
id
skill_name
activity_type
started_at
ended_at
duration_seconds
source_app
note
```

7.3 Skill

Список навыков, которые можно выбрать.

```
id
name
category
```

Примеры:

```
Python
FastAPI
SQL
React
Linux
Windows Server
Networking
```

Git
Docker
System Administration

8. Логика работы MVP

8.1 Мониторинг активного окна

Программа раз в N секунд проверяет активное окно.

Интервал проверки: 2-5 секунд

Логика:

1. Получить активное окно.
2. Получить PID процесса.
3. Получить имя процесса.
4. Получить заголовок окна.
5. Если окно не изменилось – продолжать считать время.
6. Если окно изменилось – закрыть предыдущий интервал активности.
7. Если интервал больше порога – записать/спросить навык.

8.2 Порог записи

Чтобы не мусорить данными:

Не сохранять активности короче 60 секунд.

Для разработки можно поставить:

10 секунд

8.3 Связь приложения и навыка

В MVP можно сделать ручной выбор.

Пример:

Code.exe был активен 00:32:15
Выбери навык:

1. Python
2. FastAPI
3. React
4. Git
5. Другое
6. Пропустить

Позже можно добавить правила:

```
Code.exe + .py в заголовке → Python
Code.exe + .tsx в заголовке → React
vmware.exe → System Administration / Virtualization
gns3.exe → Networking
DBeaver.exe → SQL
Obsidian.exe → Documentation / Learning Notes
```

9. Архитектура проекта

Рекомендуемая структура:

```
activity-monitor/
├── app/
│   ├── main.py
│   ├── config.py
│   └── core/
│       ├── window_tracker.py
│       ├── activity_detector.py
│       └── time_utils.py
│
│   ├── storage/
│       ├── json_storage.py
│       └── sqlite_storage.py
│
│   ├── cli/
│       └── prompts.py
│
│   ├── models/
│       ├── activity_event.py
│       ├── learning_session.py
│       └── skill.py
│
│   └── integrations/
│       └── backend_client.py
└── data/
```

```
| | | activities.json
| | | skills.json
| | |
| | | tests/
| | | requirements.txt
| | | README.md
```

10. Модули

10.1 window_tracker.py

Отвечает только за получение текущего активного окна.

Функции:

```
get_active_window_info()
```

Возвращает:

```
{
    "process_name": "Code.exe",
    "window_title": "main.py - Visual Studio Code",
    "pid": 1234
}
```

10.2 activity_detector.py

Отвечает за определение интервалов активности.

Задачи:

1. Сравнить предыдущее окно и текущее.
2. Начать новую активность при смене окна.
3. Завершить старую активность.
4. Проверить порог длительности.

10.3 prompts.py

Отвечает за вопросы пользователю.

Функции:

```
ask_skill_for_activity(activity)
ask_note_for_activity(activity)
```

10.4 json_storage.py

Первая простая версия хранения.

Функции:

```
load_skills()
save_activity(activity)
load_activities()
```

10.5 sqlite_storage.py

Вторая версия хранения.

Таблицы:

```
skills
app_activities
learning_sessions
```

10.6 backend_client.py

Позже будет отправлять данные в основной FastAPI backend.

Функции:

```
send_learning_session(session)
```

11. Этапы разработки

Этап 1. Минимальный трекер активного окна

Цель:

Научиться получать имя активного процесса и заголовок окна.

Задачи:

1. Создать отдельный проект activity-monitor.
2. Создать venv.
3. Установить pywin32 и psutil.
4. Написать функцию get_active_window_info().
5. Выводить активное окно в консоль каждые 2 секунды.

Результат:

В консоли видно:
Code.exe | main.py - Visual Studio Code
chrome.exe | FastAPI documentation
powershell.exe | Windows PowerShell

Этап 2. Подсчёт времени активности

Цель:

Определять, сколько времени пользователь провёл в одном приложении/окне.

Задачи:

1. Хранить текущее активное окно.
2. При смене окна фиксировать время начала и завершения.
3. Считать duration_seconds.
4. Игнорировать активности короче порога.

Результат:

Code.exe был активен 420 секунд
chrome.exe был активен 180 секунд

Этап 3. Ручной выбор навыка

Цель:

После завершения активности спрашивать, какой навык прокачивался.

Задачи:

1. Создать список `skills.json`.
2. После активности показывать список навыков.
3. Пользователь выбирает номер навыка.
4. Добавить короткую заметку.

Результат:

Code.exe был активен 25 минут.
Выбери навык:
1. Python
2. FastAPI
3. React
4. Git

Этап 4. Сохранение в JSON

Цель:

Сохранять сессии между запусками.

Задачи:

1. Создать `data/activities.json`.
2. Добавлять туда завершённые учебные сессии.
3. Сделать команду просмотра статистики за день.

Результат:

Сегодня:
Python — 1ч 20м
React — 40м
Git — 15м

Этап 5. SQLite

Цель:

Перейти от JSON к нормальной локальной БД.

Задачи:

1. Создать SQLite database.
2. Создать таблицы.
3. Переписать сохранение с JSON на SQLite.
4. Сделать SQL-запросы статистики.

Этап 6. Интеграция с Resume Progress Backend

Цель:

Отправлять учебные сессии в твой FastAPI backend.

Задачи:

1. Добавить backend_client.py.
2. Сделать POST /api/sessions в основном backend.
3. Отправлять завершённую учебную сессию.
4. Показывать её на dashboard.

Этап 7. Автоматические правила

Цель:

Утилита сама предлагает вероятный навык.

Примеры правил:

```
{
  "Code.exe": ["Python", "React", "FastAPI"],
  "DBeaver.exe": ["SQL", "Databases"],
  "vmware.exe": ["Virtualization", "System Administration"],
  "gns3.exe": ["Networking"],
  "Obsidian.exe": ["Documentation", "Learning Notes"]
}
```

12. Минимальные команды для старта

```
mkdir activity-monitor
cd activity-monitor
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install pywin32 psutil rich typer
pip freeze > requirements.txt
mkdir app, app\core, app\storage, app\cli, app\models, data, tests
New-Item app\main.py -ItemType File
New-Item app\core>window_tracker.py -ItemType File
New-Item app\core>activity_detector.py -ItemType File
New-Item app\storage>json_storage.py -ItemType File
New-Item app\cli>prompts.py -ItemType File
New-Item data>skills.json -ItemType File
New-Item README.md -ItemType File
```

13. Первый технический milestone

Milestone 1: Active Window Probe

Нужно реализовать программу, которая каждые 2 секунды выводит:

```
process_name | window_title | pid
```

Пример вывода:

```
Code.exe | window_tracker.py - Visual Studio Code | 12452
chrome.exe | FastAPI - Google Chrome | 28320
powershell.exe | Windows PowerShell | 8840
```

Критерии готовности:

1. Программа запускается из консоли.
2. Не падает при смене окон.
3. Корректно показывает VSCode, браузер, терминал.
4. Обработывает ситуацию, когда заголовок окна пустой.
5. Код разделён: main.py вызывает функцию из window_tracker.py.

14. Материалы для изучения

Python

Темы:

```
venv  
modules/packages  
dataclasses  
datetime  
type hints  
json  
sqlite3  
exceptions  
logging
```

Windows API через Python

Темы:

```
active window  
window handle  
process id  
process name
```

Библиотеки:

```
pywin32  
psutil
```

CLI

Темы:

```
argparse или typer  
rich для красивого вывода
```

Базы данных

Темы:

SQLite
таблицы
INSERT
SELECT
GROUP BY

Интеграция

Темы:

HTTP POST
JSON body
requests/httpx
FastAPI endpoint

15. Что НЕ делать в первой версии

Не делать GUI.
Не делать автозапуск Windows.
Не делать фоновую службу.
Не делать авторизацию.
Не делать сложную аналитику.
Не отправлять данные в интернет.
Не пытаться идеально распознавать навык автоматически.

Первая версия должна быть скучной, простой и рабочей.

16. Конечная цель проекта

В будущем утилита должна стать источником данных для Resume Progress Platform.

Итоговая связка:

Activity Monitor
↓
Learning Sessions API
↓
Backend Database
↓
Dashboard

↓
Master Resume Progress